Name – Devesh Shailesh Katneshwarkar

Reg No. – 25BAI10381

Branch – Computer Science (Artificial Intelligence and Machine Learning)

Course – Fundamentals in Artificial Intelligence and Machine Learning

Course Code – CSA 2001

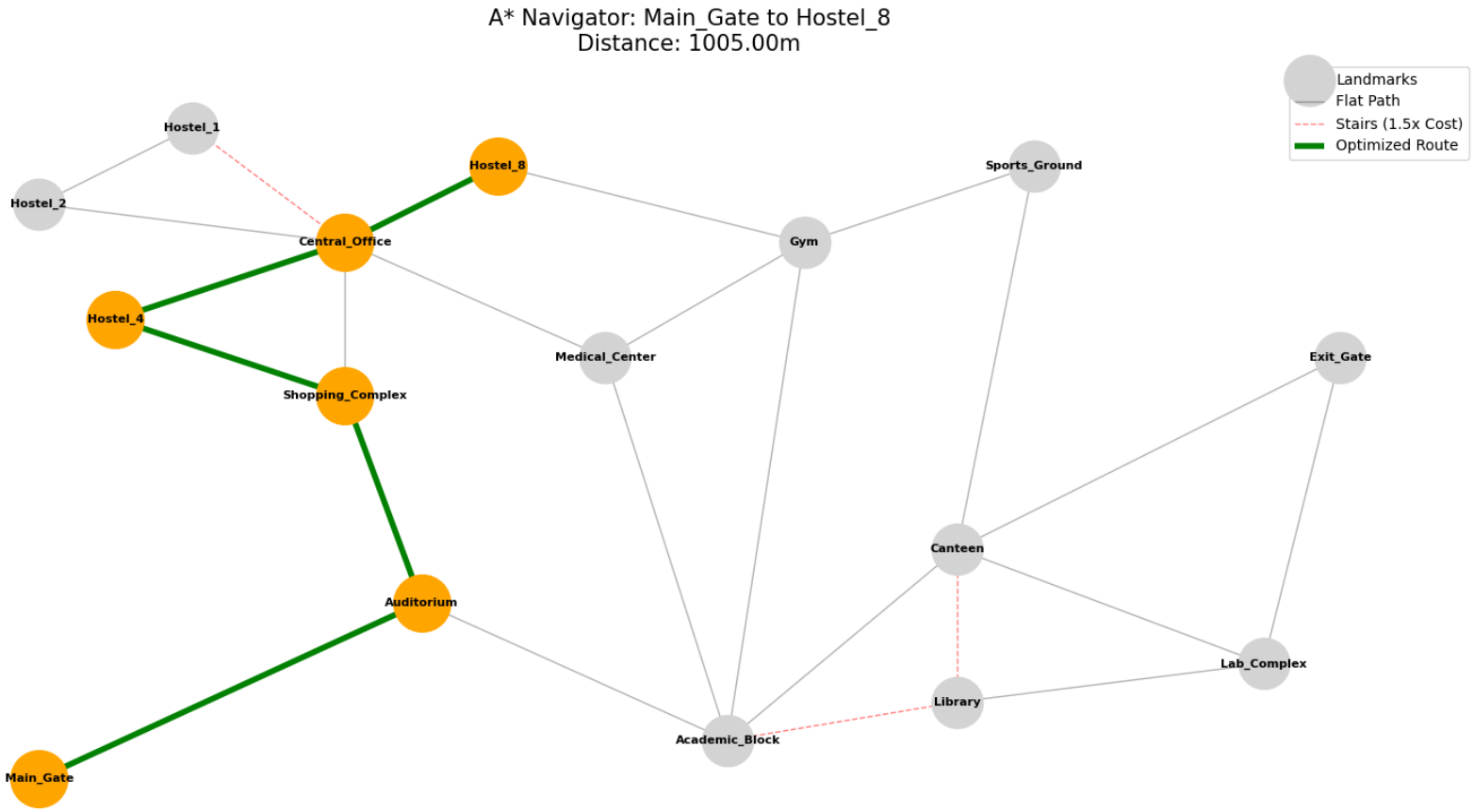
Date – 27/03/26

Project Report: *Smart Campus Navigator*

Author: *Devesh Katneshwarkar*

Domain: *Artificial Intelligence (Informed Search)*

Tools: *Python, NetworkX, Matplotlib, A* Algorithm*



1. Abstract

The **Smart Campus Navigator** is a graph-based pathfinding application designed to optimize pedestrian movement within a university campus. Unlike standard GPS tools, this system works to improve "Path Quality" specifically identifying and weighing *stairs* versus *flat* walkways. By implementing the A* Search Algorithm with a Euclidean heuristic, the project provides an optimal, computationally efficient route.

2. Problem Statement & Motivation

Navigating large, developing campuses presents unique challenges:

- **Granularity:** General maps often lack knowledge of internal walking paths between academic blocks and hostels.
- **Terrain Awareness:** Standard routers treat all paths equally, failing to account for steep inclines or stairs that might hinder someone with a heavy bag or a physical disability.
- **Purposeful Movement:** There is no existing tool that allows student-athletes to intentionally seek out challenging terrain (stairs) for daily conditioning.

3. System Architecture & AI Methodology

3.1 Graph Representation

The campus is modeled as an **Undirected Weighted Graph** G = (V, E):

- **Nodes (V):** 16 major landmarks (Main Gate, Hostels 1–8, Academic Block, etc.) are defined by (x, y) coordinates.
- **Edges (E):** Existing walkways connecting these landmarks.
- **Weights (W):** A composite cost calculated as:

Distance * Surface Factor.

3.2 The A* Search Algorithm

To find the optimal path, the project utilizes the A* algorithm, defined by the evaluation function:

$$f(n) = g(n) + h(n)$$

- **g(n) (Path Cost):** The actual cost from the start node to the current node (n).
- **h(n) (Heuristic):** To estimate cost from node (n) to the goal. We use **Euclidean Distance**:

$$h(n) = \sqrt{[(x_2 - x_1)^2 + (y_2 - y_1)^2]}$$

- This heuristic is **admissible** because a straight line is the shortest possible distance, ensuring the algorithm never overestimates and always finds the optimal path.

3.3 Innovation: Path Quality Factor

A unique multiplier is applied to the weights based on the path type:

- **Flat Paths:** Factor = 1.0
- **Staircase Paths:** Factor = 1.5 (Incentivizes the AI to avoid stairs for standard users or prioritize them for athletes).

4. Implementation Details

The project is implemented in **Python** using the following libraries:

- **NetworkX:** Used for managing the graph data structure and calculating the A* path.
- **Matplotlib:** Used for the spatial visualization of the campus map.

5. Results & Discussion

5.1 Efficiency Comparison

During testing, A* consistently outperformed uninformed search strategies:

- **Vs. BFS (Breadth-First Search)**: BFS finds the path with the fewest edges but ignores the weight (distance), often leading to longer walks.
- **Vs. Dijkstra:** While Dijkstra finds the shortest path, A* explores significantly fewer nodes by "aiming" toward the goal using the heuristic, saving computational power.

5.2 Path Analysis (For Main_Gate to Hostel_8)

When navigating from the **Main Gate** to **Hostel 8**, the algorithm evaluates the "Academic Block to Library" stairs. If the distance saved by taking the stairs does not outweigh the 1.5x cost penalty, the AI automatically reroutes the user through the "Shopping Complex" flat path.

6. Conclusion & Future Scope

The **Smart Campus Navigator** successfully demonstrates how classic AI informed search can be adapted for personalized logistics. It proves that "shortest" isn't always "best" when human factors like effort and terrain are involved.

Future Enhancements:

- **Real-time Weather Integration:** Increasing weights for unshaded paths during peak heat hours.
- **Mobile Integration:** Deploying the script as a lightweight web app for student use.
- **Multi-Goal Planning:** Optimizing routes that require stopping at the Canteen or Admin Block before heading to a hostel.

8. Final Summary

This project transitions from theoretical AI to practical applications. By mapping the university campus into a digital state-space and applying a cost-sensitive A* algorithm, we have created a tool that understands not just **where** a student needs to go, but **how** they should get there based on their physical needs and the environment.

Final Deliverables

1. **Source Code:** Python script utilizing NetworkX and Matplotlib.
2. **Digital Map:** A 16-node coordinate-based graph of the university.
3. **Heuristic Model:** A proven, admissible Euclidean distance function.